

SOCKETS: GIỚI THIỆU

The Linux Programming Interface — Michael Kerrisk

Sockets là một phương thức IPC (Inter-Process Communication) cho phép trao đổi dữ liệu giữa các ứng dụng, có thể trên cùng một host (máy tính) hoặc trên các host khác nhau được kết nối qua mạng. Triển khai rộng rãi đầu tiên của sockets API xuất hiện trong 4.2BSD năm 1983, và API này đã được port sang hầu hết mọi triển khai UNIX, cũng như phần lớn các hệ điều hành khác.

Sockets API được quy định chính thức trong POSIX.1g, được phê chuẩn năm 2000 sau khoảng một thập kỷ tồn tại ở dạng draft standard. Chuẩn này đã được thay thế bởi SUSv3.

Chương này và các chương tiếp theo mô tả cách sử dụng sockets như sau:

- Chương này cung cấp phần giới thiệu tổng quan về sockets API. Các chương tiếp theo đều giả định người đọc đã nắm được các khái niệm chung được trình bày ở đây. Chương này không đưa ra code ví dụ. Các ví dụ code trong UNIX domain và Internet domain sẽ được trình bày ở các chương sau.
- Chương 57 mô tả UNIX domain sockets, cho phép giao tiếp giữa các ứng dụng trên cùng một host.
- Chương 58 giới thiệu các khái niệm mạng máy tính và mô tả các tính năng chính của giao thức mạng TCP/IP. Đây là nền tảng cần thiết cho các chương tiếp theo.
- Chương 59 mô tả Internet domain sockets, cho phép các ứng dụng trên các host khác nhau giao tiếp qua mạng TCP/IP.
- Chương 60 thảo luận về thiết kế của các server sử dụng sockets.
- Chương 61 trình bày nhiều chủ đề nâng cao, bao gồm các tính năng bổ sung cho socket I/O, phân tích sâu hơn về giao thức TCP, và việc sử dụng socket options để truy xuất và sửa đổi các thuộc tính của sockets.

Các chương này chỉ nhằm mục đích trang bị cho người đọc nền tảng vững chắc về cách sử dụng sockets. Lập trình sockets, đặc biệt là giao tiếp mạng, là một chủ đề rất rộng và là nội dung của nhiều cuốn sách chuyên sâu. Các nguồn tham khảo thêm được liệt kê trong Mục 59.15.

56.1 Tổng quan (Overview)

Trong một kịch bản client-server điển hình, các ứng dụng giao tiếp qua sockets như sau:

- Mỗi ứng dụng tạo một socket. Một socket là "thiết bị" cho phép giao tiếp, và cả hai ứng dụng đều cần có một socket.
- Server bind socket của nó vào một địa chỉ quen thuộc (well-known address) để các client có thể tìm thấy nó.

Một socket được tạo bằng system call `socket()`, trả về một file descriptor dùng để tham chiếu đến socket trong các system call tiếp theo:

```
fd = socket(domain, type, protocol);
```

Chúng ta sẽ mô tả các domain và type của socket trong các đoạn tiếp theo. Với tất cả các ứng dụng được mô tả trong cuốn sách này, tham số protocol luôn được đặt là 0.

Communication domains (Miền giao tiếp)

Sockets tồn tại trong một communication domain, xác định:

- phương thức nhận dạng socket (tức là định dạng của "địa chỉ" socket); và
- phạm vi giao tiếp (tức là giữa các ứng dụng trên cùng host hoặc giữa các ứng dụng trên các host khác nhau kết nối qua mạng).

Các hệ điều hành hiện đại hỗ trợ ít nhất các domain sau:

- Domain `UNIX` (`AF_UNIX`) cho phép giao tiếp giữa các ứng dụng trên cùng một host. (POSIX.1g sử dụng tên `AF_LOCAL` như một synonym của `AF_UNIX`, nhưng tên này không được dùng trong SUSv3.)
- Domain `IPv4` (`AF_INET`) cho phép giao tiếp giữa các ứng dụng chạy trên các host kết nối qua mạng Internet Protocol version 4 (IPv4).
- Domain `IPv6` (`AF_INET6`) cho phép giao tiếp giữa các ứng dụng chạy trên các host kết nối qua mạng Internet Protocol version 6 (IPv6).

Mặc dù IPv6 được thiết kế như là phiên bản kế nhiệm của IPv4, giao thức IPv4 hiện vẫn đang được sử dụng rộng rãi nhất.

Bảng 56-1 tóm tắt các đặc điểm của các socket domain này.

Domain	Giao tiếp thực hiện qua	Giữa các ứng dụng	Định dạng địa chỉ	Cấu trúc địa chỉ
<code>AF_UNIX</code>	kernel	trên cùng host	pathname	<code>sockaddr_un</code>
<code>AF_INET</code>	via IPv4	trên host kết nối IPv4	32-bit IPv4 addr + 16-bit port	<code>sockaddr_in</code>
<code>AF_INET6</code>	via IPv6	trên host kết nối IPv6	128-bit IPv6 addr + 16-bit port	<code>sockaddr_in6</code>

Bảng 56-1: Socket domains

Trong một số đoạn code, chúng ta có thể thấy các hằng số có tên như `PF_UNIX` thay vì `AF_UNIX`. Trong ngữ cảnh này, AF là viết tắt của "address family" và PF là viết tắt của "protocol family". Ban đầu, người ta dự định rằng một protocol family có thể hỗ trợ nhiều address family. Trên thực tế, chưa có protocol family nào hỗ trợ nhiều address family được định nghĩa, và mọi triển khai hiện tại đều định nghĩa các hằng số PF_ là đồng nghĩa với các hằng số AF_ tương ứng. (SUSv3 chỉ định các hằng số AF_, không chỉ định các hằng số PF_.) Trong cuốn sách này, chúng ta luôn sử dụng các hằng số AF_.

Socket types (Loại socket)

Mọi triển khai socket đều cung cấp ít nhất hai loại socket: **stream** và **datagram**. Cả hai loại socket này đều được hỗ trợ trong UNIX domain và Internet domain. **Bảng 56-2** tóm tắt các thuộc tính của hai loại socket này.

Thuộc tính	Stream	Datagram
Truyền tin cậy?	Có	Không
Giữ nguyên message boundaries?	Không	Có
Connection-oriented?	Có	Không

Bảng 56-2: Các loại socket và thuộc tính của chúng

Stream sockets (`SOCK_STREAM`) cung cấp một kênh giao tiếp byte-stream tin cậy, hai chiều. Cụ thể:

- **Reliable** (tin cậy) có nghĩa là chúng ta được đảm bảo rằng dữ liệu được truyền sẽ đến nguyên vẹn tại ứng dụng nhận, đúng như khi được gửi đi (giả sử rằng không có kết nối mạng hay bên nhận bị crash), hoặc chúng ta sẽ nhận được thông báo về khả năng truyền thất bại.
- **Bidirectional** (hai chiều) có nghĩa là dữ liệu có thể được truyền theo cả hai chiều giữa hai sockets.
- **Byte-stream** có nghĩa là, giống như pipes, không có khái niệm về message boundaries (tham khảo Mục 44.1).

Một stream socket tương tự như việc sử dụng một cặp pipe để cho phép giao tiếp hai chiều giữa hai ứng dụng, với điểm khác biệt là (Internet domain) sockets cho phép giao tiếp qua mạng.

Stream sockets hoạt động theo từng cặp đã được kết nối. Vì lý do này, stream sockets được mô tả là **connection-oriented**. Thuật ngữ *peer socket* chỉ socket ở đầu kia của kết nối; *peer address* là địa chỉ của socket đó; và *peer application* là ứng dụng sử dụng peer socket. Đôi khi, thuật ngữ *remote* (hoặc *foreign*) được dùng đồng nghĩa với *peer*. Một stream socket chỉ có thể được kết nối với một peer duy nhất.

Datagram sockets (`SOCK_DGRAM`) cho phép trao đổi dữ liệu dưới dạng các tin nhắn gọi là datagrams. Với datagram sockets, message boundaries được bảo toàn, nhưng việc truyền dữ liệu không đảm bảo tin cậy. Các message có thể đến không theo thứ tự, bị trùng lặp, hoặc không đến được.

Datagram sockets là ví dụ về khái niệm tổng quát hơn là **connectionless socket**. Không giống stream socket, datagram socket không cần được kết nối với socket khác để sử dụng. (Trong Mục 56.6.2, chúng ta sẽ thấy rằng datagram sockets có thể được connect với nhau, nhưng điều này có ngữ nghĩa khác với connected stream sockets.)

Trong Internet domain, datagram sockets sử dụng giao thức UDP (User Datagram Protocol), và stream sockets (thông thường) sử dụng giao thức TCP (Transmission Control Protocol). Thay vì dùng các thuật ngữ dài như *Internet domain datagram socket* hay *Internet domain stream socket*, chúng ta thường chỉ dùng *UDP socket* và *TCP socket*.

Socket system calls

Các socket system call chính là:

- `socket()` tạo một socket mới.
- `bind()` gắn một socket vào một địa chỉ. Thông thường, server sử dụng lời gọi này để bind socket của nó vào một well-known address để các client có thể tìm thấy socket.
- `listen()` cho phép một stream socket chấp nhận các kết nối đến từ các socket khác.
- `accept()` chấp nhận một kết nối từ peer application trên một listening stream socket, và tùy chọn trả về địa chỉ của peer socket.
- `connect()` thiết lập kết nối với một socket khác.

Trên hầu hết các kiến trúc Linux (ngoại trừ Alpha và IA-64), tất cả các socket system calls thực ra được triển khai dưới dạng các library functions được ghép kênh (multiplexed) qua một system call duy nhất là `socketcall()`. (Đây là dấu tích từ giai đoạn đầu phát triển sockets trên Linux như một dự án riêng biệt.) Tuy nhiên, trong cuốn sách này chúng ta gọi tất cả các hàm này là system calls, vì đó là bản chất của chúng trong triển khai BSD gốc cũng như nhiều triển khai UNIX hiện đại khác.

Socket I/O có thể thực hiện qua các system call thông thường `read()` và `write()`, hoặc thông qua nhiều socket-specific system calls (ví dụ: `send()`, `recv()`, `sendto()`, và `recvfrom()`). Mặc định, các system calls này block nếu thao tác I/O không thể hoàn thành ngay lập tức. Nonblocking I/O cũng có thể thực hiện, bằng cách dùng thao tác `fcntl()` `F_SETFL` (Mục 5.3) để bật cờ `O_NONBLOCK` trên open file status.

Trên Linux, chúng ta có thể gọi `ioctl(fd, FIONREAD, &cnt)` để lấy số byte chưa đọc còn có sẵn trên stream socket được tham chiếu bởi file descriptor `fd`. Với datagram socket, thao tác này trả về số byte trong datagram chưa đọc kế tiếp (có thể là zero nếu datagram có độ dài bằng không), hoặc zero nếu không có datagram nào đang chờ. Tính năng này không được chỉ định trong SUSv3.

56.2 Tạo một Socket: `socket()`

System call `socket()` tạo một socket mới.

```
#include <sys/socket.h>
```

```
int socket(int domain, int type, int protocol);
```

Trả về file descriptor khi thành công, hoặc -1 khi lỗi

Tham số `domain` chỉ định communication domain cho socket. Tham số `type` chỉ định loại socket. Tham số này thường được chỉ định là `SOCK_STREAM` để tạo stream socket, hoặc `SOCK_DGRAM` để tạo datagram socket.

Tham số `protocol` luôn được đặt là 0 cho các loại socket được mô tả trong cuốn sách này. Giá trị protocol khác 0 được sử dụng với một số loại socket mà chúng ta không đề cập. Ví dụ, protocol được đặt là `IPPROTO_RAW` cho raw sockets (`SOCK_RAW`).

Khi thành công, `socket()` trả về một file descriptor dùng để tham chiếu đến socket mới được tạo trong các system calls tiếp theo.

Từ kernel 2.6.27, Linux cho phép dùng tham số `type` theo cách thứ hai, bằng cách OR hai cờ không chuẩn vào socket type. Cờ `SOCK_CLOEXEC` khiến kernel

Cấu trúc này đóng vai trò như một template cho tất cả các cấu trúc địa chỉ domain-specific. Mỗi cấu trúc địa chỉ này đều bắt đầu bằng một trường family tương ứng với trường `sa_family` của cấu trúc `sockaddr`. (Kiểu dữ liệu `sa_family_t` là kiểu số nguyên được SUSv3 quy định.) Giá trị trong trường family là đủ để xác định kích thước và định dạng của địa chỉ lưu trong phần còn lại của cấu trúc.

Một số triển khai UNIX cũng định nghĩa thêm trường `sa_len` trong cấu trúc `sockaddr` để chỉ định tổng kích thước của cấu trúc. SUSv3 không yêu cầu trường này, và nó không có trong triển khai sockets API của Linux.

Nếu định nghĩa feature test macro `_GNU_SOURCE`, thì glibc sẽ prototype các socket system call trong `<sys/socket.h>` bằng một gcc extension loại bỏ sự cần thiết phải cast (`struct sockaddr *`). Tuy nhiên, việc dựa vào tính năng này là non-portable (sẽ gây cảnh báo biên dịch trên các hệ thống khác).

56.5 Stream Sockets

Hoạt động của stream sockets có thể giải thích qua phép so sánh với hệ thống điện thoại:

1. System call `socket()` tạo ra một socket, tương đương với việc lắp đặt một điện thoại. Để hai ứng dụng có thể giao tiếp, mỗi ứng dụng phải tạo một socket.
2. Giao tiếp qua stream socket tương tự như một cuộc gọi điện thoại. Một ứng dụng phải connect socket của nó vào socket của ứng dụng khác trước khi giao tiếp có thể diễn ra. Hai sockets được kết nối như sau:
 - Một ứng dụng gọi `bind()` để bind socket vào một well-known address, sau đó gọi `listen()` để thông báo cho kernel sẵn sàng chấp nhận kết nối đến. Bước này tương tự như có một số điện thoại đã biết và đảm bảo điện thoại đang bật để người khác có thể gọi vào.
 - Ứng dụng kia thiết lập kết nối bằng cách gọi `connect()`, chỉ định địa chỉ của socket mà kết nối sẽ được thực hiện đến. Điều này tương tự như quay số điện thoại của ai đó.
 - Ứng dụng đã gọi `listen()` sau đó chấp nhận kết nối bằng `accept()`. Điều này tương tự như nhắc máy khi chuông reo. Nếu `accept()` được thực hiện trước khi peer application gọi `connect()`, thì `accept()` sẽ block ("chờ bên điện thoại").
3. Sau khi kết nối được thiết lập, dữ liệu có thể được truyền theo cả hai chiều giữa các ứng dụng (tương tự như cuộc hội thoại hai chiều qua điện thoại) cho đến khi một trong hai đóng kết nối bằng `close()`. Giao tiếp được thực hiện qua system call `read()` và `write()` thông thường hoặc qua nhiều socket-specific system calls (như `send()` và `recv()`) cung cấp thêm chức năng.

Hình 56-1 minh họa việc sử dụng các system calls dùng với stream sockets.

Passive socket (server)	Active socket (client)
<code>socket()</code>	<code>socket()</code>
<code>bind()</code>	
<code>listen()</code>	
<code>accept()</code>	<code><-- block đến khi connect()</code> <code>client kết nối --></code>
(Nhiều lần) truyền dữ liệu hai chiều	
<code>read()</code>	<code>write()</code>

<code>write()</code>	<code>read()</code>
<code>close()</code>	<code>close()</code>

Hình 56-1: Tổng quan các system calls dùng với stream sockets

Active socket và passive socket

Stream sockets thường được phân biệt là active hoặc passive:

- Mặc định, một socket được tạo bằng `socket()` là **active**. Một active socket có thể được dùng trong lời gọi `connect()` để thiết lập kết nối tới một passive socket. Điều này được gọi là thực hiện *active open*.
- Một **passive socket** (còn được gọi là *listening socket*) là socket đã được đánh dấu để cho phép kết nối đến bằng cách gọi `listen()`. Chấp nhận một kết nối đến được gọi là thực hiện *passive open*.

Trong hầu hết các ứng dụng dùng stream sockets, server thực hiện passive open và client thực hiện active open. Chúng ta ngầm định kịch bản này trong các phần tiếp theo, nên thay vì nói "ứng dụng thực hiện active socket open", chúng ta thường chỉ nói "client". Tương tự, "server" được đồng nhất với "ứng dụng thực hiện passive socket open".

56.5.1 Lắng nghe kết nối đến: `listen()`

System call `listen()` đánh dấu stream socket được tham chiếu bởi file descriptor `sockfd` là passive. Socket sau đó sẽ được dùng để chấp nhận kết nối từ các socket khác (active).

```
#include <sys/socket.h>

int listen(int sockfd, int backlog);
```

Trả về 0 khi thành công, hoặc -1 khi lỗi

Không thể áp dụng `listen()` lên một connected socket — tức là socket mà `connect()` đã được thực hiện thành công, hoặc socket được trả về từ lời gọi `accept()`.

Để hiểu mục đích của tham số `backlog`, cần lưu ý rằng client có thể gọi `connect()` trước khi server gọi `accept()`. Điều này có thể xảy ra, ví dụ, vì server đang bận xử lý client khác. Điều này tạo ra một *pending connection* (kết nối đang chờ), như minh họa trong Hình 56-2.

Passive socket (server)	Active socket (client)
<code>socket()</code>	<code>socket()</code>
<code>bind()</code>	
<code>listen()</code>	
	<code>connect()</code>
	(có thể block nếu quá nhiều pending connection requests)
<code>accept()</code>	

Hình 56-2: Một pending socket connection

Kernel phải ghi lại một số thông tin về mỗi pending connection request để `accept()` tiếp theo có thể xử lý được. Tham số `backlog` cho phép chúng ta giới hạn số lượng pending connections như vậy. Các yêu cầu kết nối trong phạm vi giới hạn này thành công ngay lập tức. (Với TCP sockets, chi tiết phức tạp hơn một chút, như sẽ thấy trong Mục 61.6.4.) Các yêu cầu kết nối

vượt quá giới hạn sẽ block cho đến khi một pending connection được chấp nhận (qua `accept()`) và được xóa khỏi hàng đợi.

SUSv3 cho phép triển khai đặt giới hạn trên cho giá trị `backlog`, và có thể lặn lẽ làm tròn xuống giới hạn đó. SUSv3 quy định rằng triển khai nên công bố giới hạn này bằng cách định nghĩa hằng số `SOMAXCONN` trong `<sys/socket.h>`. Trên Linux, hằng số này được định nghĩa với giá trị 128. Tuy nhiên, từ kernel 2.4.25, Linux cho phép điều chỉnh giới hạn này lúc runtime thông qua file Linux-specific `/proc/sys/net/core/somaxconn`. (Trong các phiên bản kernel trước, giới hạn `SOMAXCONN` là bất biến.) Trong triển khai BSD sockets gốc, giới hạn trên của `backlog` là 5, và chúng ta có thể thấy con số này trong các đoạn code cũ. Tất cả các triển khai hiện đại đều cho phép giá trị `backlog` cao hơn, cần thiết cho các network servers dùng TCP sockets để phục vụ nhiều client.

56.5.2 Chấp nhận kết nối: `accept()`

System call `accept()` chấp nhận một kết nối đến trên listening stream socket được tham chiếu bởi file descriptor `sockfd`. Nếu không có pending connections khi `accept()` được gọi, lời gọi sẽ block cho đến khi có yêu cầu kết nối đến.

```
#include <sys/socket.h>
```

```
int accept(int sockfd, struct sockaddr *addr, socklen_t *addrlen);
```

Trả về file descriptor khi thành công, hoặc -1 khi lỗi

Điều quan trọng cần hiểu về `accept()` là nó tạo ra một socket **mới**, và chính socket mới này được kết nối với peer socket đã thực hiện `connect()`. Một file descriptor cho connected socket mới được trả về là kết quả hàm của lời gọi `accept()`. Listening socket (`sockfd`) vẫn mở và có thể dùng để chấp nhận thêm kết nối. Một server application điển hình tạo một listening socket, bind nó vào well-known address, sau đó xử lý tất cả yêu cầu client bằng cách chấp nhận kết nối qua socket đó.

Các tham số còn lại của `accept()` trả về địa chỉ của peer socket. Tham số `addr` trỏ đến một cấu trúc dùng để lưu địa chỉ socket. Tham số `addrlen` là một value-result argument — nó trỏ đến một số nguyên mà trước khi gọi, phải được khởi tạo bằng kích thước của buffer mà `addr` trỏ tới; sau khi trả về, số nguyên này sẽ cho biết số byte dữ liệu thực sự được sao chép vào buffer.

Nếu không quan tâm đến địa chỉ của peer socket, thì `addr` và `addrlen` nên được đặt lần lượt là `NULL` và 0. (Nếu muốn, có thể lấy địa chỉ của peer sau này bằng system call `getpeername()`, như mô tả trong Mục 61.5.)

Từ kernel 2.6.28, Linux hỗ trợ một system call mới không chuẩn là `accept4()`. System call này thực hiện cùng tác vụ như `accept()`, nhưng hỗ trợ thêm tham số `flags` để sửa đổi hành vi của system call. Hai cờ được hỗ trợ: `SOCK_CLOEXEC` và `SOCK_NONBLOCK`, với ngữ nghĩa tương tự như khi dùng trong `socket()`.

56.5.3 Kết nối đến Peer Socket: `connect()`

System call `connect()` kết nối active socket được tham chiếu bởi file descriptor `sockfd` tới listening socket có địa chỉ được chỉ định bởi `addr` và `addrlen`.

```
#include <sys/socket.h>
```

```
int connect(int sockfd, const struct sockaddr *addr, socklen_t addrlen);
```

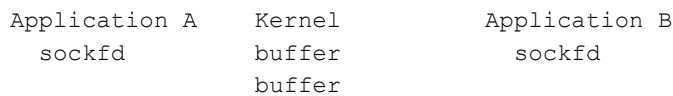
Trả về 0 khi thành công, hoặc -1 khi lỗi

Các tham số `addr` và `addrlen` được chỉ định theo cùng cách với các tham số tương ứng trong `bind()`.

Nếu `connect()` thất bại và chúng ta muốn thử lại kết nối, thì SUSv3 quy định cách thức portable là đóng socket, tạo socket mới, và thử kết nối lại với socket mới.

56.5.4 I/O trên Stream Sockets

Một cặp connected stream sockets cung cấp một kênh giao tiếp hai chiều giữa hai đầu cuối. Hình 56-3 cho thấy điều này trong UNIX domain.



Hình 56-3: UNIX domain stream sockets cung cấp kênh giao tiếp hai chiều

Ngữ nghĩa của I/O trên connected stream sockets tương tự như đối với pipes:

- Để thực hiện I/O, chúng ta dùng system calls `read()` và `write()` (hoặc socket-specific `send()` và `recv()`, được mô tả trong Mục 61.3). Vì sockets là bidirectional, cả hai lời gọi đều có thể dùng ở mỗi đầu của kết nối.
- Một socket có thể được đóng bằng system call `close()` hoặc do ứng dụng kết thúc. Sau đó, khi peer application cố đọc từ đầu kia của kết nối, nó sẽ nhận end-of-file (sau khi tất cả dữ liệu đã được buffered được đọc). Nếu peer application cố ghi vào socket của nó, nó sẽ nhận tín hiệu `SIGPIPE`, và system call thất bại với lỗi `EPIPE`. Như đã lưu ý trong Mục 44.2, cách thông thường để xử lý trường hợp này là bỏ qua tín hiệu `SIGPIPE` và nhận biết về kết nối đã đóng qua lỗi `EPIPE`.

56.5.5 Kết thúc kết nối: `close()`

Cách thông thường để kết thúc một stream socket connection là gọi `close()`. Nếu nhiều file descriptor tham chiếu đến cùng một socket, thì kết nối chỉ được kết thúc khi tất cả các descriptor đó được đóng.

Giả sử rằng sau khi chúng ta đóng kết nối, ứng dụng peer bị crash hoặc không đọc/xử lý đúng dữ liệu mà chúng ta đã gửi. Trong trường hợp đó, chúng ta không có cách nào biết rằng đã xảy ra lỗi. Nếu cần đảm bảo rằng dữ liệu đã được đọc và xử lý thành công, chúng ta phải xây dựng một loại acknowledgement protocol vào ứng dụng. Điều này thường bao gồm một acknowledgement message tường minh được gửi lại từ peer.

Trong Mục 61.2, chúng ta mô tả system call `shutdown()`, cung cấp khả năng kiểm soát tinh tế hơn về cách đóng stream socket connection.

56.6 Datagram Sockets

Hoạt động của datagram sockets có thể giải thích qua phép so sánh với hệ thống bưu chính:

4. System call `socket()` tương đương với việc lắp đặt một hộp thư. Mỗi ứng dụng muốn gửi hoặc nhận datagrams đều tạo một datagram socket bằng `socket()`.

- Để cho phép ứng dụng khác gửi datagram đến, một ứng dụng dùng `bind()` để bind socket của nó vào một well-known address. Thông thường, server bind socket của nó vào well-known address, và client khởi tạo giao tiếp bằng cách gửi datagram đến địa chỉ đó. (Trong một số domain — đặc biệt là UNIX domain — client cũng có thể cần dùng `bind()` để gán địa chỉ cho socket của nó nếu muốn nhận datagrams được gửi bởi server.)
- Để gửi datagram, ứng dụng gọi `sendto()`, nhận vào một trong các tham số là địa chỉ socket mà datagram sẽ được gửi đến. Điều này tương tự như ghi địa chỉ người nhận lên phong bì và gửi đi.
- Để nhận datagram, ứng dụng gọi `recvfrom()`, có thể block nếu chưa có datagram nào đến. Vì `recvfrom()` cho phép lấy địa chỉ của người gửi, chúng ta có thể gửi phản hồi nếu muốn.
- Khi socket không còn cần thiết, ứng dụng đóng nó bằng `close()`.

Cũng như hệ thống bưu chính, khi nhiều datagrams (thư) được gửi từ địa chỉ này đến địa chỉ khác, không có gì đảm bảo rằng chúng sẽ đến theo thứ tự đã gửi, hoặc thậm chí đến được. Datagrams có thêm một khả năng không có trong hệ thống bưu chính: vì các giao thức mạng bên dưới đôi khi có thể truyền lại một data packet, cùng một datagram có thể đến nhiều hơn một lần.

Hình 56-4 minh họa việc sử dụng các system calls dùng với datagram sockets.

Server	Client
<code>socket()</code>	<code>socket()</code>
<code>bind()</code>	
(Nhiều lần) truyền dữ liệu hai chiều	
<code>recvfrom()</code>	<code>sendto()</code>
<code>sendto()</code>	<code>recvfrom()</code>
<code>close()</code>	<code>close()</code>

Hình 56-4: Tổng quan các system calls dùng với datagram sockets

56.6.1 Trao đổi Datagrams: `recvfrom()` và `sendto()`

Các system calls `recvfrom()` và `sendto()` nhận và gửi datagrams trên datagram socket.

```
#include <sys/socket.h>
```

```
ssize_t recvfrom(int sockfd, void *buffer, size_t length, int flags,
                 struct sockaddr *src_addr, socklen_t *addrlen);
    Trả về số byte nhận được, 0 khi EOF, hoặc -1 khi lỗi
```

```
ssize_t sendto(int sockfd, const void *buffer, size_t length, int flags,
               const struct sockaddr *dest_addr, socklen_t addrlen);
    Trả về số byte đã gửi, hoặc -1 khi lỗi
```

Giá trị trả về và ba tham số đầu tiên của các system calls này giống như đối với `read()` và `write()`.

Tham số thứ tư, `flags`, là một bit mask điều khiển các tính năng socket I/O. Chúng ta sẽ đề cập đến các tính năng này khi mô tả system calls `recv()` và `send()` trong Mục 61.3. Nếu không cần bất kỳ tính năng nào trong số đó, có thể đặt `flags` là 0.

Các tham số `src_addr` và `addrlen` được dùng để lấy hoặc chỉ định địa chỉ của peer socket mà chúng ta đang giao tiếp.

Với `recvfrom()`, `src_addr` và `addrlen` trả về địa chỉ của remote socket đã gửi datagram. `src_addr` là con trỏ đến cấu trúc địa chỉ phù hợp với communication domain. Cũng như `accept()`, `addrlen` là value-result argument: trước lời gọi, phải khởi tạo bằng kích thước của cấu trúc mà `src_addr` trỏ tới; sau khi trả về, nó chứa số byte thực sự được ghi vào cấu trúc đó.

Nếu không quan tâm đến địa chỉ người gửi, có thể đặt cả `src_addr` và `addrlen` là `NULL`. Trong trường hợp này, `recvfrom()` tương đương với việc dùng `recv()` để nhận datagram. Chúng ta cũng có thể dùng `read()` để đọc datagram, tương đương với dùng `recv()` với tham số `flags` là 0.

Bất kể giá trị được chỉ định cho `length`, `recvfrom()` lấy đúng một message từ datagram socket. Nếu kích thước message vượt quá `length` bytes, message sẽ bị cắt bớt lạng lẽ xuống còn `length` bytes.

Nếu dùng system call `recvmsg()` (Mục 61.13.2), có thể biết về datagram bị cắt qua cờ `MSG_TRUNC` được trả về trong trường `msg_flags` của cấu trúc `msghdr` được trả về.

Với `sendto()`, các tham số `dest_addr` và `addrlen` chỉ định socket mà datagram sẽ được gửi đến. `dest_addr` là cấu trúc địa chỉ phù hợp với communication domain, được khởi tạo với địa chỉ của destination socket. Tham số `addrlen` chỉ định kích thước của `dest_addr`.

Trên Linux, có thể dùng `sendto()` để gửi datagrams có độ dài bằng 0. Tuy nhiên, không phải tất cả các triển khai UNIX đều cho phép điều này.

56.6.2 Sử dụng `connect()` với Datagram Sockets

Mặc dù datagram sockets là connectionless, system call `connect()` vẫn có tác dụng khi áp dụng lên datagram sockets. Gọi `connect()` trên datagram socket khiến kernel ghi lại một địa chỉ cụ thể làm peer của socket đó. Thuật ngữ *connected datagram socket* được dùng cho socket như vậy. Thuật ngữ *unconnected datagram socket* được dùng cho datagram socket chưa gọi `connect()` (tức là trạng thái mặc định của datagram socket mới).

Sau khi datagram socket đã được connect:

- Datagrams có thể được gửi qua socket bằng `write()` (hoặc `send()`) và tự động được gửi đến cùng peer socket. Cũng như `sendto()`, mỗi lời gọi `write()` tạo ra một datagram riêng.
- Chỉ có datagrams được gửi bởi peer socket mới có thể được đọc trên socket này.

Lưu ý rằng tác động của `connect()` là bất đối xứng với datagram sockets. Các phát biểu trên chỉ áp dụng cho socket đã gọi `connect()`, không áp dụng cho remote socket mà nó được kết nối đến (trừ khi peer application cũng gọi `connect()` trên socket của nó).

Chúng ta có thể thay đổi peer của connected datagram socket bằng cách gọi thêm `connect()`. Cũng có thể xóa bỏ peer association bằng cách chỉ định cấu trúc địa chỉ trong đó address family (ví dụ: trường `sun_family` trong UNIX domain) được đặt là `AF_UNSPEC`. Tuy nhiên, lưu ý rằng nhiều triển khai UNIX khác không hỗ trợ dùng `AF_UNSPEC` cho mục đích này.

SUSv3 khá mơ hồ về việc xóa peer associations, chỉ nói rằng một kết nối có thể được reset bằng cách gọi `connect()` chỉ định "null address", mà không định nghĩa thuật ngữ này. SUSv4 chỉ định tường minh việc sử dụng `AF_UNSPEC`.

Ưu điểm rõ ràng của việc đặt peer cho datagram socket là có thể dùng các socket system calls I/O đơn giản hơn khi truyền dữ liệu. Chúng ta không còn cần dùng `sendto()` với các tham số `dest_addr` và `addrlen`, mà có thể dùng `write()` thay thế. Việc đặt peer hữu ích chủ yếu trong ứng dụng cần gửi nhiều datagrams đến cùng một peer (điển hình ở một số datagram clients).

Trên một số triển khai TCP/IP, kết nối datagram socket tới peer mang lại cải thiện hiệu năng. Trên Linux, việc connect datagram socket không tạo ra nhiều sự khác biệt về hiệu năng.

56.7 Tóm tắt (Summary)

Sockets cho phép giao tiếp giữa các ứng dụng trên cùng host hoặc trên các host khác nhau kết nối qua mạng.

Một socket tồn tại trong một communication domain, xác định phạm vi giao tiếp và định dạng địa chỉ dùng để nhận dạng socket. SUSv3 chỉ định các communication domain UNIX (`AF_UNIX`), IPv4 (`AF_INET`), và IPv6 (`AF_INET6`).

Hầu hết các ứng dụng dùng một trong hai loại socket: stream hoặc datagram. Stream sockets (`SOCK_STREAM`) cung cấp một kênh giao tiếp byte-stream tin cậy, hai chiều giữa hai đầu cuối. Datagram sockets (`SOCK_DGRAM`) cung cấp giao tiếp không tin cậy, connectionless, hướng message.

Một stream socket server điển hình tạo socket bằng `socket()`, sau đó bind socket vào well-known address bằng `bind()`. Server sau đó gọi `listen()` để cho phép nhận kết nối trên socket. Mỗi client connection được chấp nhận trên listening socket bằng `accept()`, trả về file descriptor cho socket mới được kết nối với client socket. Một stream socket client điển hình tạo socket bằng `socket()`, sau đó thiết lập kết nối bằng `connect()`, chỉ định well-known address của server. Sau khi hai stream sockets được kết nối, dữ liệu có thể được truyền theo cả hai chiều bằng `read()` và `write()`. Khi tất cả các process có file descriptor tham chiếu đến stream socket endpoint thực hiện `close()` tường minh hoặc ngầm định, kết nối được kết thúc.

Một datagram socket server điển hình tạo socket bằng `socket()`, sau đó bind socket vào well-known address bằng `bind()`. Vì datagram sockets là connectionless, socket của server có thể được dùng để nhận datagrams từ bất kỳ client nào. Datagrams có thể được nhận bằng `read()` hoặc socket-specific `recvfrom()`, trả về địa chỉ của sending socket. Một datagram socket client tạo socket bằng `socket()`, sau đó dùng `sendto()` để gửi datagram đến địa chỉ được chỉ định (tức là địa chỉ của server). System call `connect()` có thể được dùng với datagram socket để đặt peer address cho socket. Sau đó, không còn cần chỉ định destination address cho datagrams gửi đi; có thể dùng `write()` để gửi datagram.

Đọc thêm: Tham khảo các nguồn thông tin được liệt kê trong Mục 59.15.